SOLID – D: Dependency Inversion Principle (DIP)

Plano e documentação base para o repositório que ilustra o **Princípio da Inversão de Dependência**.

> **Nota sobre a sigla:** na ordem do SOLID, a Inversão de Dependência é o **D** (quinto princípio). O **I** é *Interface Segregation* (Segregação de Interfaces), que ainda não foi planejado na série. Sugestão de numeração: `solid-04-isp` (a fazer) e `solid-05-dip` (este documento).

1. Objetivo do repositório

Mostrar como **inverter o sentido das dependências** para que as regras de negócio (alto nível) não dependam de detalhes de infraestrutura (baixo nível), como banco de dados, e-mail ou APIs externas. O repositório implementa o mesmo sistema em duas versões:

- **Antes:** o caso de uso instancia e usa diretamente classes concretas de banco e de e-mail (`new MySqlRepository()`, `new SmtpClient()`).
- **Depois:** o caso de uso depende de **abstrações** que ele mesmo define; a infraestrutura implementa essas abstrações e é conectada na raiz da aplicação.

O ganho a evidenciar: regras de negócio testáveis sem infraestrutura, e infraestrutura trocável sem tocar no domínio.

2. O que é o DIP

> A. Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações. B. Abstrações não devem depender de detalhes. Detalhes devem depender de abstrações. — Robert C. Martin

A "inversão" em uma imagem



Sem o DIP, a seta de dependência aponta do negócio para o detalhe. Com o DIP, o detalhe passa a apontar para a abstração definida pelo negócio: a dependência foi **invertida**.

Três conceitos que costumam ser confundidos

Conceito	O que é	Relação com o DIP
DIP (Dependency Inversion Principle)	Princípio de design: depender de abstrações	O **objetivo**
DI (Dependency Injection)	Técnica: receber dependências de fora (construtor, parâmetro)	Uma **forma** de aplicar o DIP
IoC (Inversion of Control) / contêiner de DI	Quem controla a criação dos objetos (framework)	Uma **ferramenta**, não obrigatória

Pontos que costumam gerar confusão

- **Injetar dependência não é o mesmo que inverter.** Receber uma classe concreta pelo construtor ainda é depender do detalhe.
- **Quem é dono da interface?** A abstração deve pertencer ao módulo de **alto nível** (o consumidor), e não ao de baixo nível. Se a interface mora na camada de infraestrutura, a dependência não foi realmente invertida.
- **Contêiner de DI não é requisito.** Montar os objetos manualmente em um *composition root* já aplica o princípio.
- **Relação com os outros princípios:** o DIP depende de abstrações confiáveis (**LSP**), pequenas e coesas (**ISP**), e viabiliza a extensão sem modificação (**OCP**).

3. Como identificar uma violação (code smells)

- `new` de classes de infraestrutura dentro de regras de negócio (`new MySqlConnection()`, `new SmtpClient()`).
- Imports de pacotes de baixo nível (driver de banco, SDK de nuvem, cliente HTTP) dentro do domínio ou do caso de uso.
- Chamadas estáticas e *singletons* acessados de qualquer lugar (`Database.getInstance()`).
- Testes unitários que exigem banco real, rede ou arquivos para validar uma regra simples.
- Trocar a tecnologia (MySQL para PostgreSQL, SMTP para API de e-mail) obriga a editar o código de negócio.
- Dependências circulares entre pacotes de negócio e infraestrutura.

4. Cenário proposto: finalização do pedido

Reaproveitar o domínio da série (**Pedido, Cliente, Item**) e o caso de uso `FinalizarPedido` do repositório do SRP, agora com foco em **quem depende de quem**:

Módulo	Nível	Depende de
`FinalizarPedido` (caso de uso)	Alto	Abstrações de persistência e notificação
`MySqlPedidoRepository`	Baixo	Driver do banco
`SmtpNotificador`	Baixo	Biblioteca de e-mail

4.1 Versão que VIOLA o DIP

```
import { MySqlPedidoRepository } from "../infra/MySqlPedidoRepository";
import { SmtpNotificador } from "../infra/SmtpNotificador";

class FinalizarPedido {
  private repo = new MySqlPedidoRepository();   // acoplado ao detalhe
  private notificador = new SmtpNotificador();  // acoplado ao detalhe

  executar(pedido: Pedido): void {
    this.repo.salvar(pedido);
    this.notificador.enviar(pedido.cliente.email, "Pedido confirmado");
  }
}
```

Por que é um problema? O caso de uso, que é a regra de negócio, **conhece e instancia** MySQL e SMTP. Para testá-lo é preciso um banco e um servidor de e-mail; para trocar o banco é preciso editar o negócio.

4.2 Versão que RESPEITA o DIP

```
// application/ports.ts  → as interfaces pertencem ao núcleo (alto nível)
export interface PedidoRepository {
  salvar(pedido: Pedido): void;
}
export interface Notificador {
  notificar(destino: string, mensagem: string): void;
}

// application/FinalizarPedido.ts
class FinalizarPedido {
  constructor(
    private repo: PedidoRepository,
    private notificador: Notificador,
  ) {}

  executar(pedido: Pedido): void {
    this.repo.salvar(pedido);
    this.notificador.notificar(pedido.cliente.email, "Pedido confirmado");
  }
}

// infra/MySqlPedidoRepository.ts  → o detalhe depende da abstração
class MySqlPedidoRepository implements PedidoRepository { /* ... */ }
class SmtpNotificador implements Notificador { /* ... */ }

// main.ts  → composition root: único lugar que conhece as classes concretas
const caso = new FinalizarPedido(
  new MySqlPedidoRepository(),
  new SmtpNotificador(),
);
```

Agora `FinalizarPedido` não importa nada de `infra`. Quem conhece as implementações concretas é apenas o ponto de montagem (`main`).

> Os exemplos estão em TypeScript apenas como referência; adapte para a linguagem escolhida.

4.3 Segunda demonstração: testes sem infraestrutura

Mostrar que o ganho aparece nos testes, usando **fakes simples** escritos à mão, sem biblioteca de mock:

```
class RepositorioEmMemoria implements PedidoRepository {
  salvos: Pedido[] = [];
  salvar(p: Pedido) { this.salvos.push(p); }
}

class NotificadorFalso implements Notificador {
  enviadas: string[] = [];
  notificar(destino: string, msg: string) { this.enviadas.push(destino); }
}

test("finalizar pedido salva e notifica", () => {
  const repo = new RepositorioEmMemoria();
  const notif = new NotificadorFalso();

  new FinalizarPedido(repo, notif).executar(pedidoExemplo);

  expect(repo.salvos).toHaveLength(1);
  expect(notif.enviadas).toEqual(["cliente@email.com"]);
});
```

5. Técnicas para aplicar o DIP

Técnica	Quando usar	Exemplo
Injeção por construtor	Dependência obrigatória do objeto	`FinalizarPedido(repo, notificador)`
Portas e adaptadores (Hexagonal)	Isolar o núcleo do mundo externo	Interfaces no núcleo, adaptadores na borda
Composition root	Concentrar a criação dos objetos concretos	`main.ts` que monta o grafo de objetos
Fakes e stubs em testes	Testar o núcleo isoladamente	`RepositorioEmMemoria`
Contêiner de DI (opcional)	Muitos objetos para montar	Spring, NestJS, .NET DI etc.

O repositório deve focar em **injeção por construtor + composition root**; o contêiner de DI pode ser citado como extra.

6. Estrutura sugerida do repositório

```
solid-05-dip/
├── README.md
├── docs/
│   └── dip.md                  # este documento
├── before/                     # caso de uso acoplado à infraestrutura
│   ├── src/
│   └── tests/
├── after/                      # dependências invertidas
│   ├── src/
│   │   ├── domain/             # Pedido, Item, Cliente
│   │   ├── application/        # FinalizarPedido + ports (interfaces)
│   │   ├── infra/              # adaptadores: MySql..., Smtp..., InMemory...
│   │   └── main.ts             # composition root
│   └── tests/
│       ├── unit/               # núcleo com fakes, sem infraestrutura
│       └── integration/        # adaptadores reais (opcional)
└── CHANGELOG.md
```

Regra de dependência entre pastas: `domain` não importa nada; `application` importa só `domain`; `infra` importa `application` e `domain`; `main` importa tudo.

7. Roteiro de implementação (plano de trabalho)

1. **Base:** copiar o domínio e o caso de uso da série (Pedido, Cliente, Item, `FinalizarPedido`).
2. **Versão `before`:** implementar `FinalizarPedido` com `new` das classes concretas de banco e e-mail.
3. **Demonstrar a dor:** tentar escrever um teste unitário da versão `before` e registrar no `CHANGELOG` o que foi preciso (banco, servidor SMTP, tempo de execução).
4. **Definir as portas:** criar `PedidoRepository` e `Notificador` na camada `application`.
5. **Refatorar para `after`:** receber as portas pelo construtor e mover as classes concretas para `infra`.
6. **Composition root:** criar `main` montando o grafo de objetos.
7. **Testes:** unitários com fakes em memória; integração opcional com os adaptadores reais.

8. **Provar a troca de tecnologia:** adicionar um segundo adaptador (ex.: `PostgresPedidoRepository` ou `ApiEmailNotificador`) alterando **apenas** o `main`.
9. **Verificar a regra de dependência:** conferir (manualmente ou com ferramenta de análise) que `domain` e `application` não importam `infra`.
10. **Documentar:** README com o comparativo antes vs. depois e como rodar.

8. Requisitos de implementação

- Mesmo comportamento externo nas duas versões (os testes de aceitação passam em ambas).
- Em `after`, nenhum arquivo de `domain` ou `application` pode importar algo de `infra`.
- As interfaces (portas) ficam na camada de **alto nível**, não na infraestrutura.
- Dependências recebidas pelo construtor; sem `new` de infraestrutura fora do `main`.
- Sem *singletons* globais nem *service locator* para obter dependências.
- Testes unitários do núcleo rodam **sem** banco, rede ou arquivos.
- Interfaces pequenas e coesas (ponte para o **I** do SOLID).
- Sem frameworks de DI obrigatórios; o foco é o princípio.

9. Métricas para provar o ganho

Métrica	Antes	Depois
Imports de `infra` dentro de `domain` / `application`	1 ou mais	0
Testes unitários que exigem banco ou rede	Todos	0
Tempo de execução dos testes do caso de uso	Segundos	Milissegundos
Arquivos de negócio alterados ao trocar banco ou e-mail	1 ou mais	0 (só `main`)

10. Cuidados: o outro extremo

- **Abstração em excesso:** não crie interface para tudo. Tipos estáveis e da própria linguagem (`String`, `List`, `Date`) não precisam de inversão. Inverta nas **fronteiras voláteis**: banco, rede, arquivos, relógio, serviços externos.

- **Interface espelho:** uma interface que apenas copia todos os métodos de uma classe concreta, com um único implementador, costuma ser ruído. Desenhe a porta pelo que o **consumidor precisa**.
- **Service locator disfarçado:** buscar dependências em um registro global esconde o acoplamento e dificulta os testes; prefira injeção explícita.
- **Contêiner de DI como muleta:** usar o framework sem entender o princípio leva a dependências ocultas e configuração mágica.
- **Interface no lugar errado:** se a porta mora em `infra`, o núcleo ainda depende da infraestrutura.

11. Checklist de revisão

- [ ] O caso de uso depende apenas de abstrações que ele mesmo define?
- [ ] As interfaces ficam na camada de alto nível, e não na infraestrutura?
- [ ] `domain` e `application` não importam nada de `infra`?
- [ ] O único lugar com `new` de classes concretas de infraestrutura é o `main`?
- [ ] Os testes do núcleo rodam sem banco, rede ou arquivos?
- [ ] Trocar de tecnologia exige mudança apenas no `main` e no novo adaptador?
- [ ] Não há *singletons* ou *service locator* escondendo dependências?
- [ ] As interfaces são pequenas e feitas sob medida para o consumidor?
- [ ] O README explica o problema, a solução e como rodar?

12. Modelo de README do repositório

1. **Título e objetivo** — qual princípio é ilustrado.
2. **Conceito** — definição do DIP em poucas linhas, diferenciando DIP, DI e IoC.
3. **Problema** — a dor de `before` (trecho com `new` de infraestrutura e o teste impossível de isolar).
4. **Solução** — como `after` resolve (diagrama com o sentido das setas de dependência antes e depois).
5. **Como executar** — instalação, build e testes (unitários e integração).
6. **Comparativo** — tabela de métricas.
7. **Referências.**

13. Referências

- Robert C. Martin — "The Dependency Inversion Principle" (C++ Report, 1996).

- Robert C. Martin — *Agile Software Development: Principles, Patterns, and Practices* e *Clean Architecture* (regra da dependência).
- Alistair Cockburn — "Hexagonal Architecture" (portas e adaptadores).
- Martin Fowler — "Inversion of Control Containers and the Dependency Injection pattern".
- Mark Seemann — *Dependency Injection: Principles, Practices, and Patterns* (composition root).

14. Próximo passo

Como o **I – Interface Segregation** ainda está pendente na série, ele é o próximo candidato natural: o DIP depende de interfaces pequenas e coesas, e o ISP mostra como desenhá-las. Ao concluir os cinco repositórios, vale um repositório final integrado (`solid-all`), em que o sistema de pedidos aplica S, O, L, I e D juntos.